

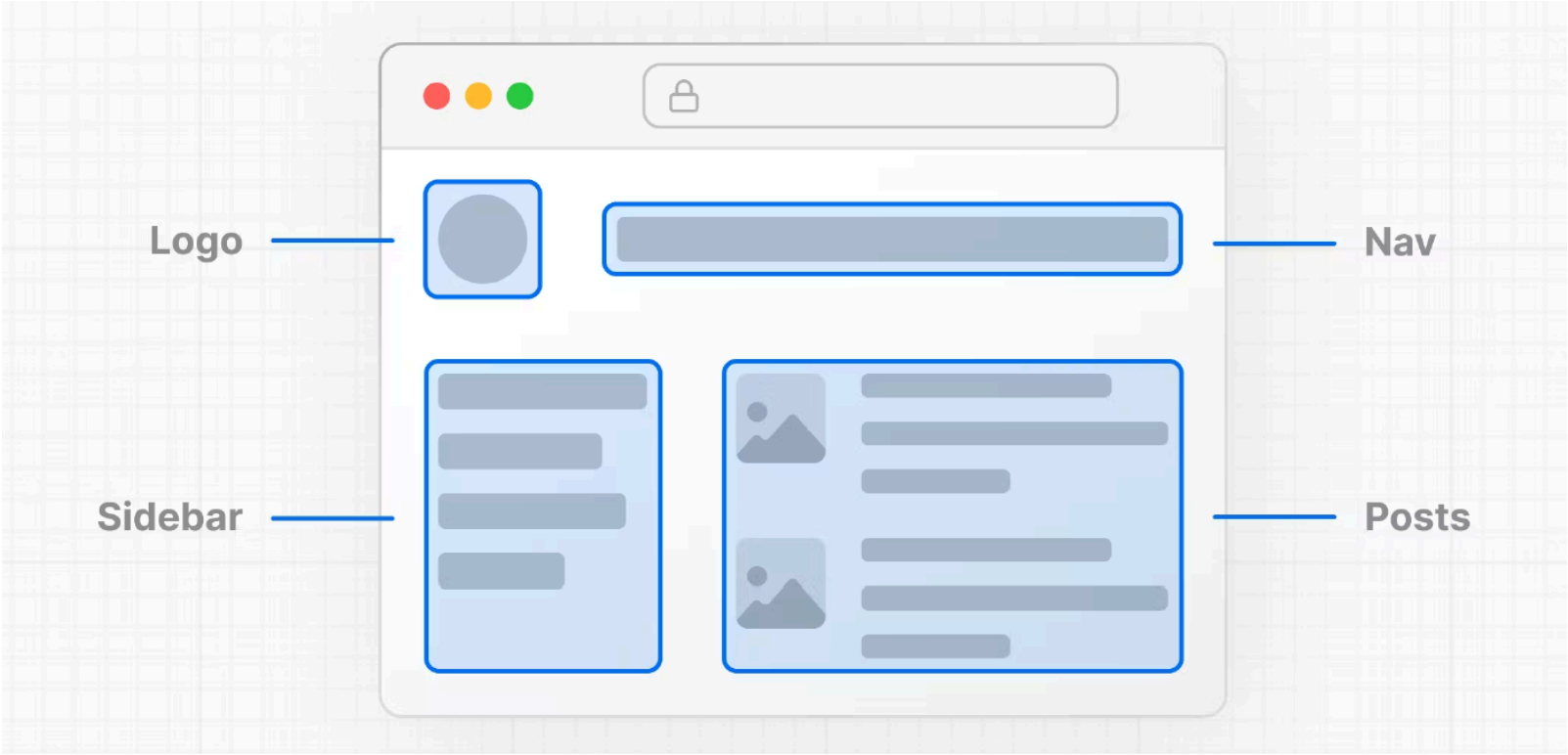
# React Intro

# Building Blocks Of A Web Application

- `User Interface` → how users will consume and interact with your application.
- `Routing` → how users navigate between different parts of your application.
- `Data Fetching` → where your data lives and how to get it.
- `Rendering` → when and where you render static or dynamic content.
- `Integrations` → what third-party services you use (for CMS, auth, payments, etc.) and how you connect to them.
- `Infrastructure` → where you deploy, store, and run your application code (serverless, CDN, edge, etc.).
- `Performance` → how to optimize your application for end-users.
- `Scalability` → how your application adapts as your team, data, and traffic grow.

# → What Is React ❓

- React is a JavaScript library for building interactive **user interfaces**.




> 📑https://react.dev/learn🔗

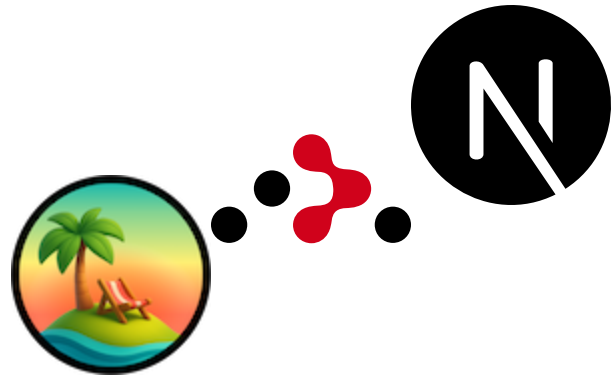
# → Build A React App

## from Scratch

- Install a build tool
  - Vite, Parcel, ...
- Build Common Application Patterns
  - Routing
    - what content or pages to display when a user visits a particular URL
  - Data Fetching
    - fetching and caching data from data source e.g. APIs
  - Code-splitting
    - splitting your code into smaller chunks that can be loaded on demand
  - Rendering
    - server-side rendering (SSR)
    - client-side rendering (CSR)

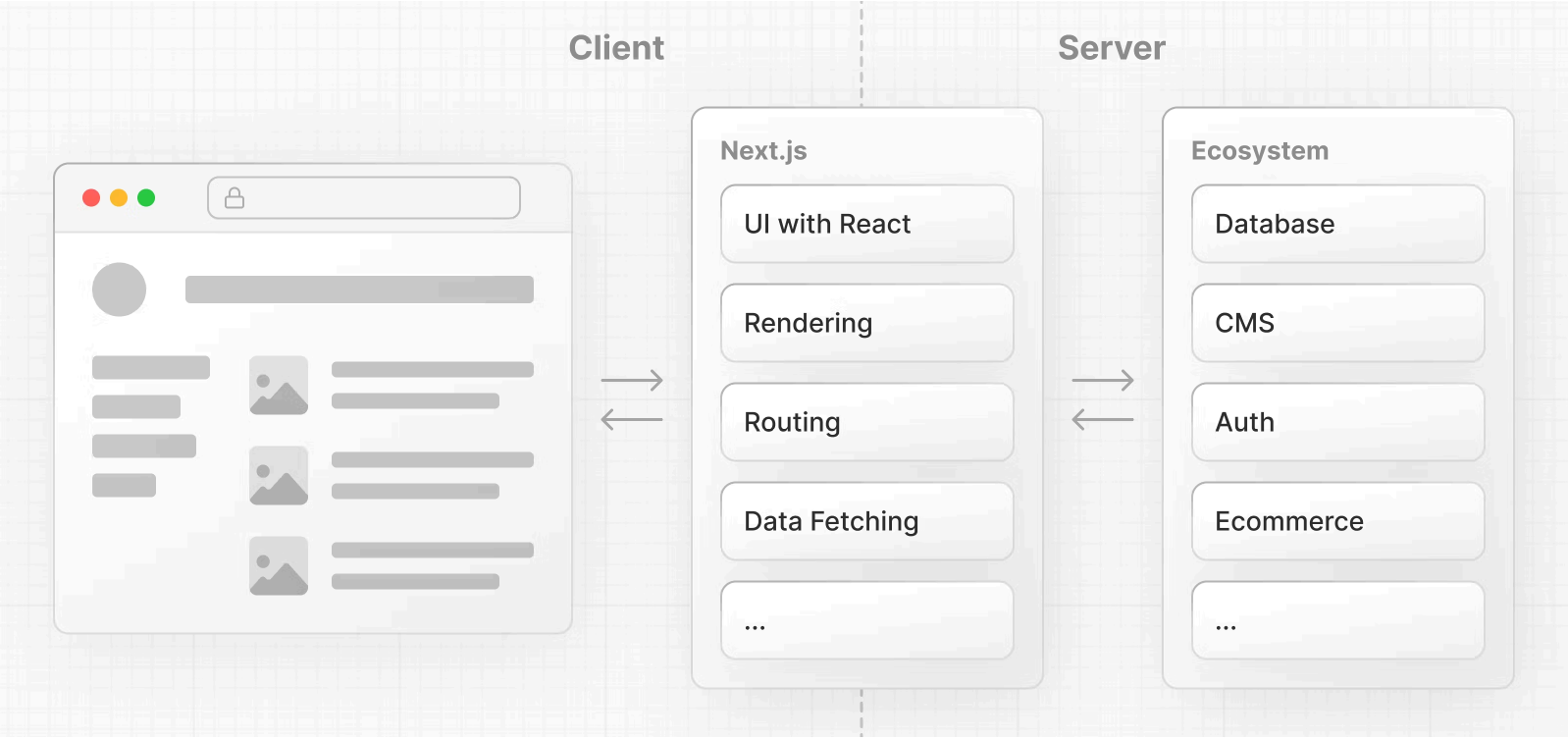
## with a framework

- Next.js
- React Router
- Tanstack
- ...



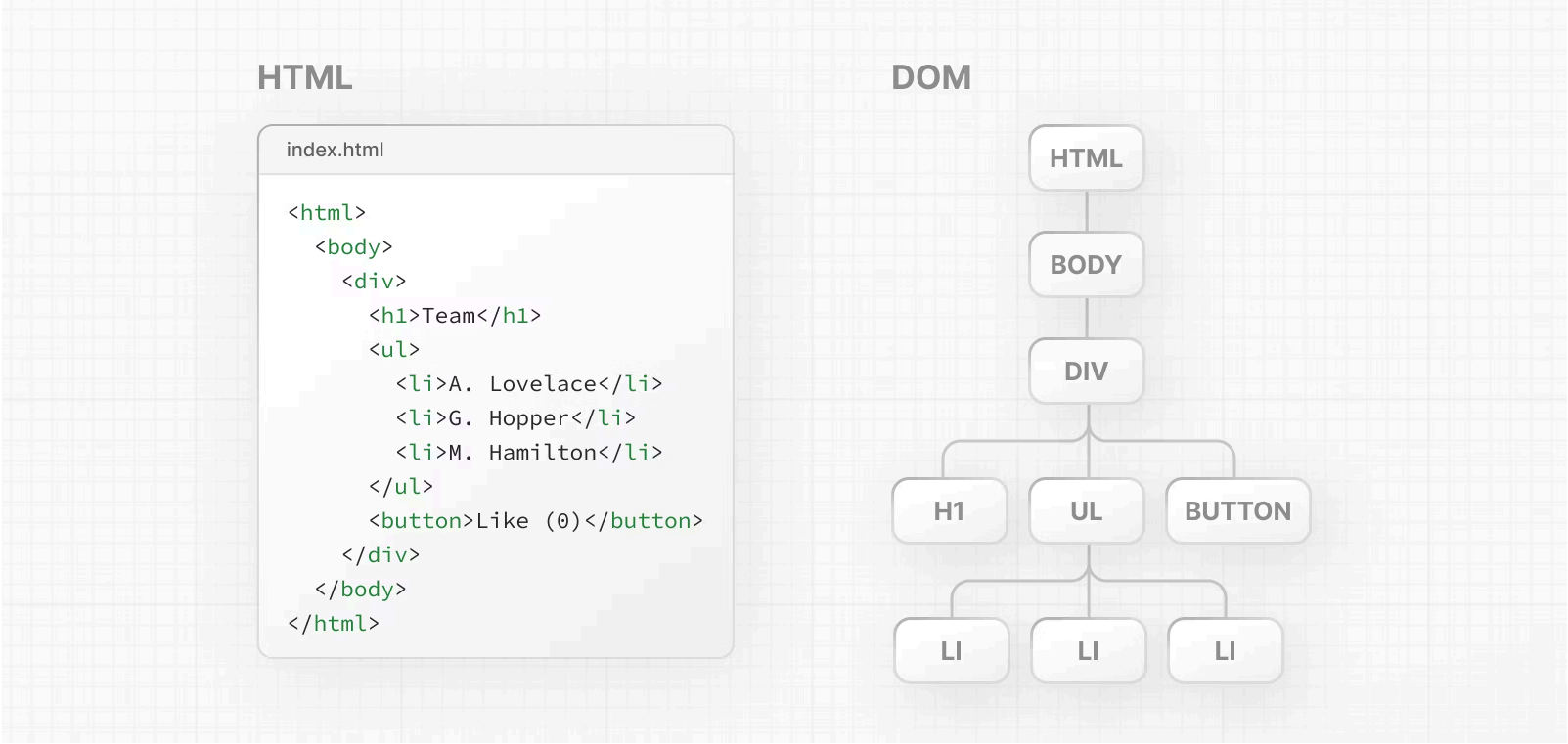
# → What Is Next.Js ❓

- Next.js is a React framework that gives you building blocks to **create web applications**.



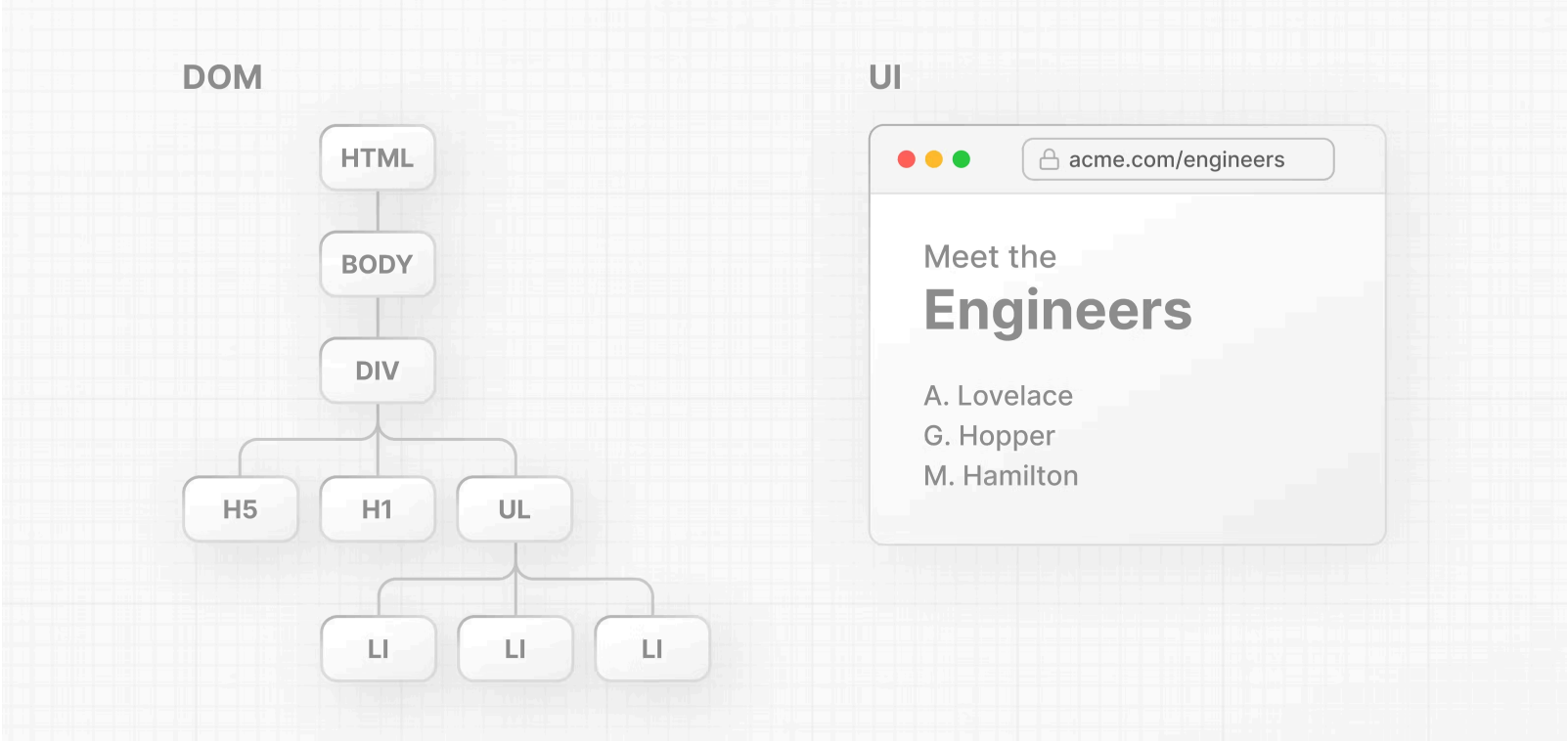

> 📑https://nextjs.org/learn🔗

# → Rendering User Interfaces (UI)




- When a user visits a web page, the server returns an HTML file.
- The browser reads the HTML and constructs the **Document Object Model (DOM)**.

# → Rendering User Interfaces (UI)




- **DOM** → object representation of the HTML elements.
  - tree-like structure with parent and child relationships
  - bridge between code and the user interface

> 📑 Introduction to the DOM🔗

# → Updating UI With Javascript

```
<html><body>
    <div id="app"></div>
</body></html>
```

```
<html><body>
    <div id="app"></div>
    <script type="text/javascript">
       // Select the div element with 'app' id
      const app = document.getElementById('app');
    </script>
</body></html>
```

```
<html><body>
    <div id="app"></div>
    <script type="text/javascript">
      const app = document.getElementById('app');
      const header = document.createElement('h1');// Create a new H1 element
      const text = 'Develop. Preview. Ship.';// Create a new text node for the H1 element
      const headerContent = document.createTextNode(text);
      header.appendChild(headerContent);// Append the text to the H1 element
      app.appendChild(header);// Place the H1 element inside the div
    </script>
  </body></html>
```

# → Rendering User Interfaces (UI)

**DOM**

Developer Tools Elements

```
<html>
  <head></head>
  <body>
    <div id="app">
      <h1>Develop. Preview. Ship. 🚀</h1>
    </div>
    <script type="text/javascript">...</script>
  </body>
</html>
```

**Source Code (HTML)**

index.html

```
<html>
  <head></head>
  <body>
    <div id="app"></div>
    <script type="text/javascript">...</script>
  </body>
</html>
```

- HTML → initial page content
- DOM → updated page content

# → Getting Started With React

```
<html><body><div id="app"></div>
<script src="https://unpkg.com/@babel/standalone/babel.min.js"></script>
<script type="text/babel" data-type="module">
  import React from 'https://esm.sh/react@19';
  import ReactDOM from 'https://esm.sh/react-dom@19/client';
  const domNode = document.getElementById("app");
  const root = ReactDOM.createRoot(domNode);
  root.render(<h1>Develop. Preview. Ship.</h1>);
</script></html>
```

> react → the core React library.
>
> react-dom → DOM-specific methods that enable you to use React with the DOM.

- text/babel → JSX → syntax extension for JavaScript / describe UI in a HTML-like syntax
- Babel → a JavaScript compiler / transforms JSX code into regular JavaScript



# → Getting Started With React

## React code

```
<script type="text/babel" data-type="module">
  import React from 'https://esm.sh/react@19';
  import ReactDOM from 'https://esm.sh/react-dom@19/client';
  const domNode = document.getElementById("app");
  const root = ReactDOM.createRoot(domNode);
  root.render(<h1>Develop. Preview. Ship.</h1>);
</script>
```

## JavaScript code

```
<script type="text/javascript">
  const app = document.getElementById('app');
  const header = document.createElement('h1');
  const text = 'Develop. Preview. Ship.';
  const headerContent = document.createTextNode(text);
  header.appendChild(headerContent);
  app.appendChild(header);
</script>
```

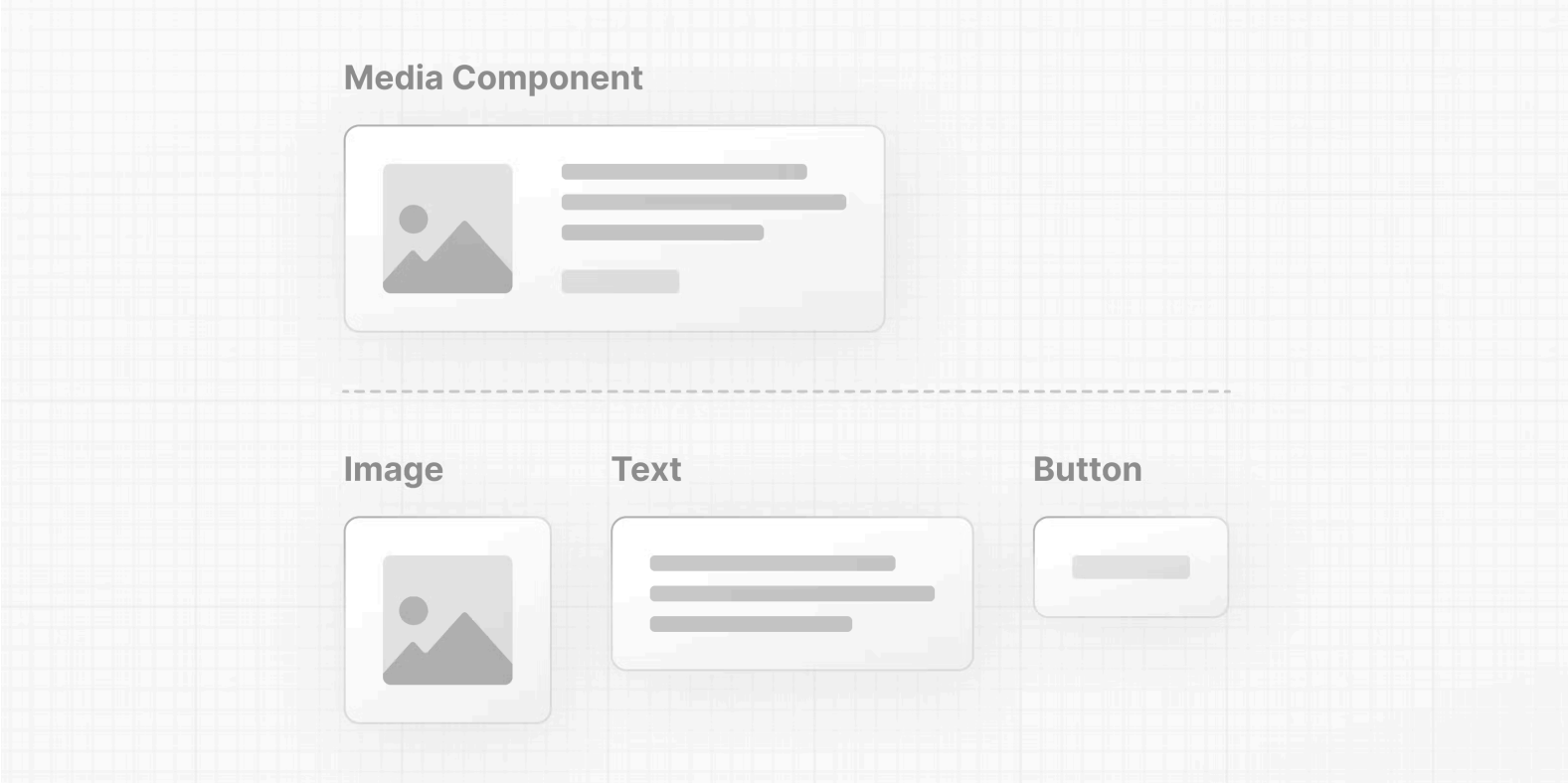


# → Essential JavaScript For React

- Functions and Arrow Functions
- Objects
- Arrays and array methods
- Destructuring
- Template literals
- Ternary Operators
- ES Modules and Import / Export Syntax

# → Building UI With Components

- React core concepts:
  - **Components**
    - self-contained, reusable snippets of code
    - **Props**
    - **State**



# → Building UI With Components

- Creating components
  - A component is a function that returns UI elements

```
<script type="text/babel" data-type="module">
  import React from 'https://esm.sh/react@19';
  import ReactDOM from 'https://esm.sh/react-dom@19/client';
  const app = document.getElementById("app")

  function Header() {
    return <h1>Develop. Preview. Ship.</h1>;
  }

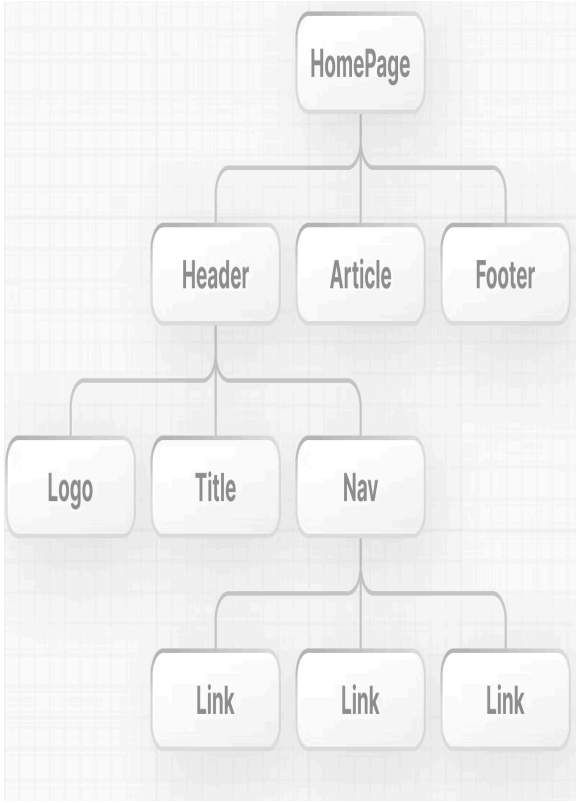
  const root = ReactDOM.createRoot(app);
  root.render(<Header />);
</script>
```

> ℹ️ React DOM Components 🔗

## Nesting components

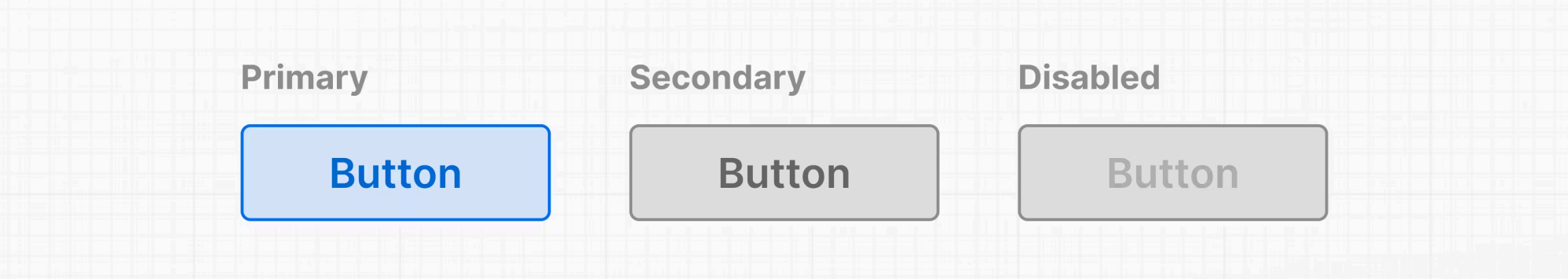
```
<script type="text/babel" data-type="module">
import ...
const app = document.getElementById("app")
function Header() {
  return <h1>Develop. Preview. Ship.</h1>;
}
function HomePage() {
 return (
   <div><Header /></div>
);
}
const root = ReactDOM.createRoot(app);
root.render(<HomePage />);
</script>
```

## Component trees




> ℹ Components can render other components, but you must never nest their definitions

# → Displaying Data With Props




- components can accept custom arguments (or props)
  - change the component's behavior
  - pass down props from parent components to child components

> ℹ Props are the information that you pass to a babel tag

> ℹ React → data flows down the component tree (one-way data flow)

# → Displaying Data With Props

```
<script type="text/babel" data-type="module">
const app = document.getElementById("app")
function Header(props) {
  console.log(props); // { title: "React" }
  return <h1>Develop. Preview. Ship.</h1>;
}

function HomePage() {
 return (
   <div>
    <Header title="React" />
   </div>
);
}
const root = ReactDOM.createRoot(app);
root.render(<HomePage />);
</script>
```

```
<script type="text/babel" data-type="module">
const app = document.getElementById("app")
function Header({ title }) {
  console.log(title);
  return <h1>{title}</h1>;
}

function HomePage() {
 return (
   <div>
    <Header title="React" />
   </div>
);
}
const root = ReactDOM.createRoot(app);
root.render(<HomePage />);
</script>
```

> { } → allows for JavaScript expression

# → Displaying Data With Props

- object property

```
function Header(props) {
  return <h1>{props.title}</h1>;
}
```

- template literal

```
function Header({ title }) {
  return <h1>{`Cool ${title}`}</h1>;
}
```

- returned value of a function

```
function createTitle(title) {
  if (title) { return title;}
  else {return 'Default title';}
}
function Header({ title }) {
  return <h1>{createTitle(title)}</h1>;
}
```

- ternary operators

```
function Header({ title }) {
  return <h1>{title ? title : 'Default Title'}</h1>;
}
```

# → Displaying Data With Props

```
function Header({ title }) {
  return <h1>{title ? title : 'Default title'}</h1>;
}

function HomePage() {
  return (
    <div>
      <Header title="React" />
      <Header title="A new title" />
    </div>
  );
}
```

**React**

**A new title**

# → Displaying Data With Props

- Iterating through lists

```
function HomePage() {
  const names = ['Ada Lovelace',
                 'Grace Hopper',
                 'Margaret Hamilton'];
   return (
    <div>
      <Header title="Develop. Preview. Ship." />
      <ul>
        {names.map((name) => (
          <li>{name}</li>
        ))}
      </ul>
    </div>
  );
}
```

```
function HomePage() {
  const names = ['Ada Lovelace',
                 'Grace Hopper',
                 'Margaret Hamilton'];
   return (
    <div>
      <Header title="Develop. Preview. Ship." />
      <ul>
        {names.map((name) => (
          <li key={name}>{name}</li>
        ))}
      </ul>
    </div>
  );
}
```

> React needs something to uniquely identify items in an array so it knows which elements to update in the DOM.

# → Listening & Handling Events

```
function HomePage() {
  const names = ['Ada Lovelace', 'Grace Hopper', 'Margaret Hamilton'];

  function handleClick() {    // Handling events
    alert("like button clicked")
  }

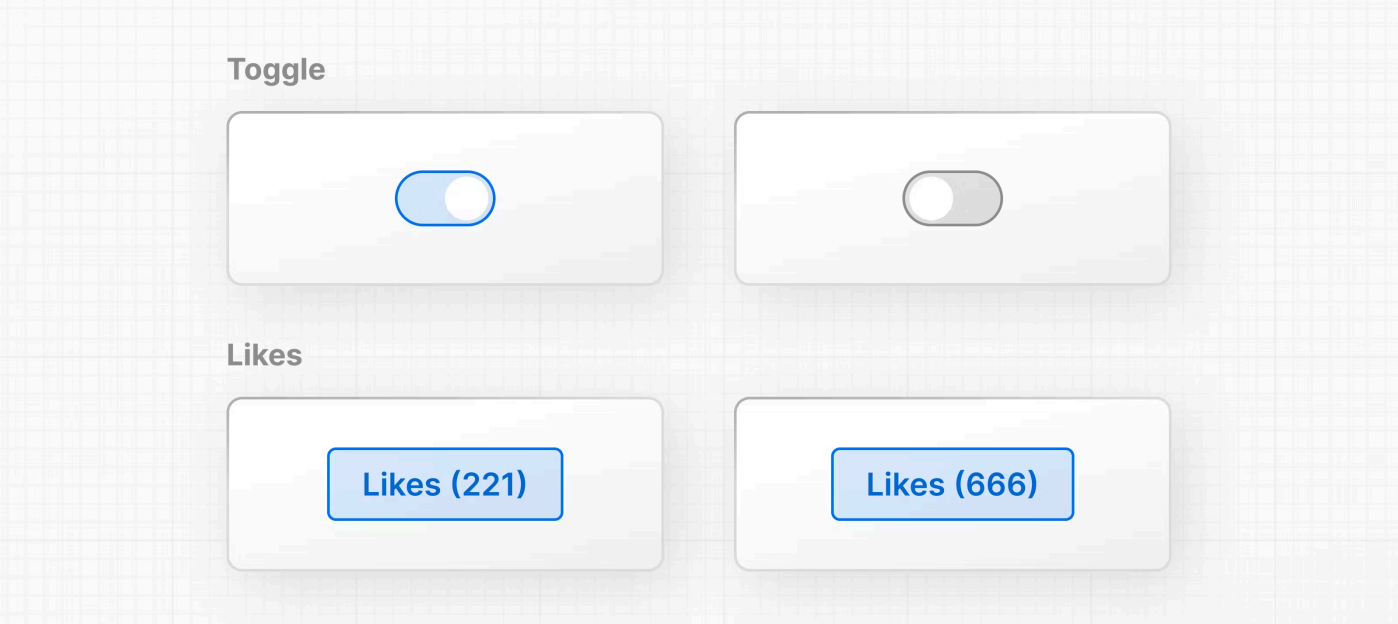
  return (
    <div>
      <Header title="Develop. Preview. Ship." />
      <ul>
        {names.map((name) => (
          <li key={name}>{name}</li>
        ))}
      </ul>
      {/* Listening to events */}
      <button onClick={handleClick}>Like</button>
    </div>
  );
}
```

# → Fragment (<>...</>)

**`<Fragment>`** → group elements without a wrapper node

```
<script type="text/babel" data-type="module">
import ...
const app = document.getElementById("app")
function HomePage() {
 return (
    <>
      <Post title="An update" body="It's been a while since I posted..." />
      <Post title="My new blog" body="I am starting a new blog!" />
    </>
  )
}
function Post({ title, body }) {
  return (
    <>
      <PostTitle title={title} /><PostBody body={body} />
    </>
  );
}
function PostTitle({ title }) { return <h1>{title}</h1>}
function PostBody({ body }) {  return ( <article><p>{body}</p></article> ); }
const root = ReactDOM.createRoot(app);
root.render(<HomePage />);
```

# → State And Hooks




- `State` → your component may need to “remember” some information
- `Hooks` → set of functions to manage the state of your components

# → Adding Interactivity With State

```
function HomePage() {
  const names = ['Ada Lovelace', 'Grace Hopper', 'Margaret Hamilton'];

  const [likes, setLikes] = React.useState(0);
  // likes --> the state value
  // setLikes -->  a function to update the value
  // useState(0) --> initial value of your likes state

  function handleClick() {    // Handling events
    alert("like button clicked")
    setLikes(likes + 1);
  }

  return (
    <div>
      <Header title="Develop. Preview. Ship." />
      <ul> {names.map((name) => (<li key={name}>{name}</li>))} </ul>
      {/* Listening to events */}
      <button onClick={handleClick}>Like({likes})</button>
    </div>
  );
}
```



# → Adding Interactivity With State

> What is the difference between props and state ?

- props → passed to components as the first function parameter (read-only)
- state → initiated and stored within a component (can change over time)
- You can pass the state information to children components as props
- Logic for updating the state should be kept within the component where state was initially created.

> State: A Component's Memory

# → Setting Up A React Environment

- Node.js

```
node -v
```

- Vite

```
npm install -g create-vite
```

- Create a new React app with Vite

```
create-vite my-app --template react
```

```
◇  Install with npm and start now?
│  Yes
│
◇  Scaffolding project in d:\tmp\my-app...
│
◇  Installing dependencies with npm...

added 151 packages, and audited 152 packages in 5s

36 packages are looking for funding
  run `npm fund` for details

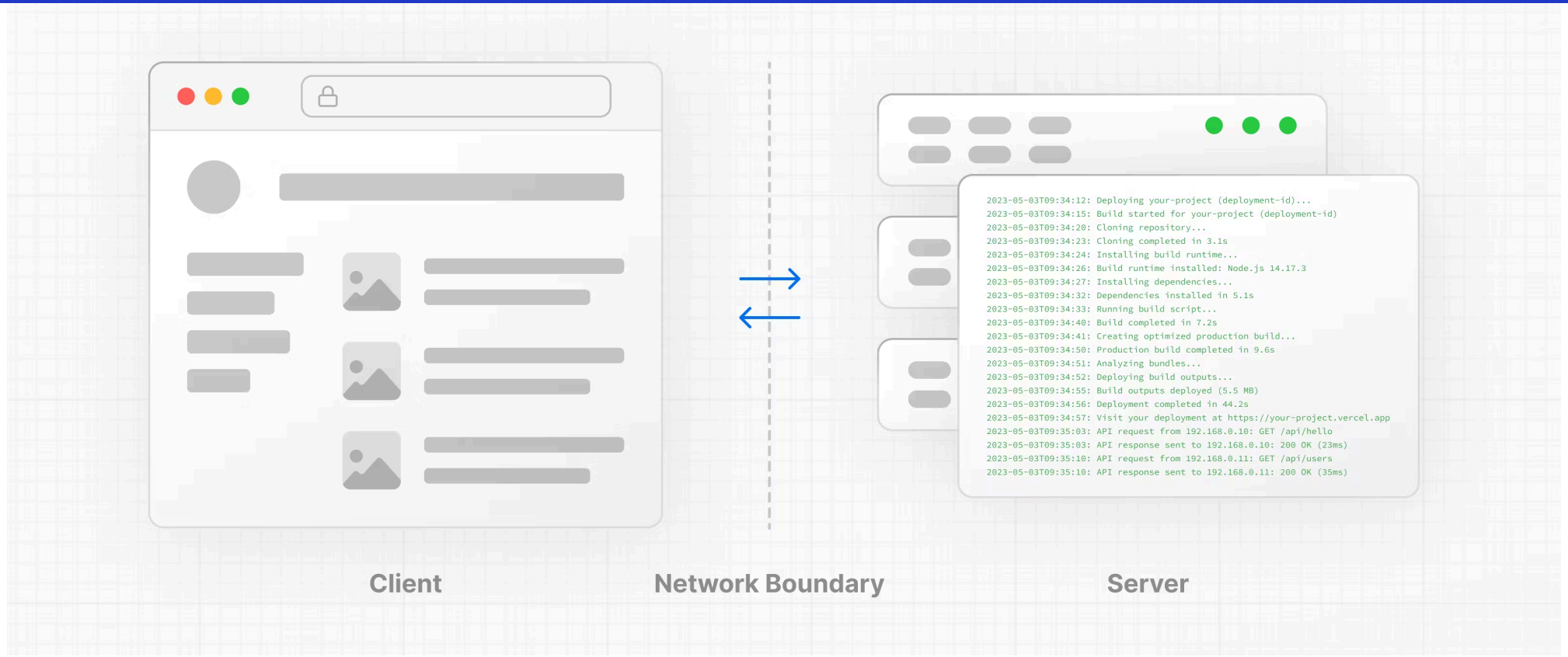
found 0 vulnerabilities
│
◇  Starting dev server...

> my-app@0.0.0 dev
> vite


  VITE v8.0.8  ready in 1532 ms

  ➜  Local:   http://localhost:5173/
  ➜  Network: use --host to expose
```

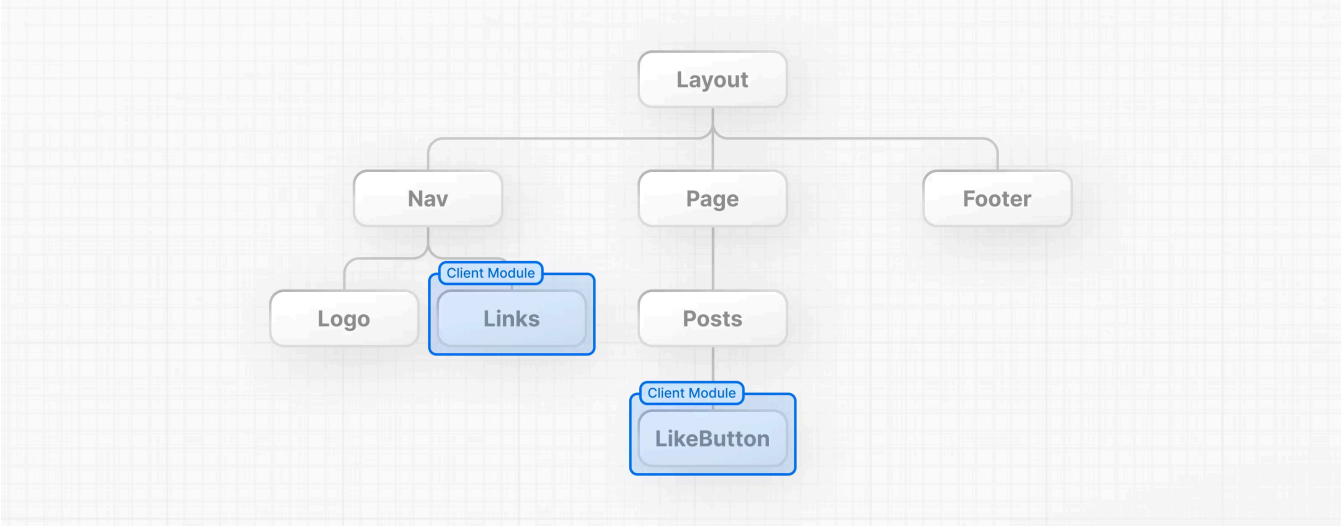
# → Server And Client Environments




- Certain operations are better suited for one environment over the other
  - Server → rendering and data fetching
  - Client → interactive UI - DOM update.

# → Network Boundary

- Network Boundary → a conceptual line that separates the different environments
  - you choose where to place the network boundary in your component tree
  - The "use client" & "use server" Directives
    - fetch data and render a user's posts on the server (Server Components)
    - render the interactive LikeButton for each post on the client ( Client Components)





# → Server And Client Components

- Two module graphs
  - **Server module graph** (or tree) contains all the Server Components that are rendered on the server
  - **Client module graph** (or tree) contains all Client Components
- **React Server Component Payload** (RSC) is sent to the client containing:
  - The rendered result of Server Components
  - Placeholders (or holes) for where Client Components should be rendered and references to their JavaScript files.



# → Thank You!

## Ερωτήσεις ?